Source: docs\CATALOG_RECOVERY.md | Export: 11 November 2026

Catalog recovery and administrator navigation - local preparation

Reporting index correction (22 October): see REPORT_INDEX_AUDIT.md. The seven orders indexes are now confirmed READY. The earlier aggregate field order below was incorrect; four exact backend-requested definitions have been appended locally. Do not recreate the seven READY indexes or add the four unused historical aggregate definitions.

Evidence and scope

- Browse: supplied request c6dfa185-2f8e-426a-9c4d-e34c72fd9982, action browse, phase operation, reason firestore-index-required, code 9. Its status/featured/createdAt query cannot use only the older status/createdAt index.
- Dashboard: independently supplied request a93e17e6-238e-4204-866c-176b69d9475a (2026-10-22 12:43:02.799), action reports:get, phase operation, same reason/code. Reporting aggregates have separate index requirements. The precise failing member of the eight report queries was not in that log; no index-creation URL was supplied.
- The reviewed index file already contained the browse definitions below and reporting definitions that were later found to have the wrong aggregate suffix order. Three narrow order-count indexes were added for the new reporting-completeness guard; all previous definitions and field overrides are preserved. Logs prove the deployed queries lack usable required indexes, not whether indexes are absent or still building. A successful different catalog request does not verify browse or reports.
- No production requests, data writes, index/rule publication or deployment were performed. Owner product 4uHf6KMAGq091NArs1L5 was not read or modified by tooling. All mutation fixtures use loopback emulators.

Exact index manifest

All entries below use queryScope COLLECTION (not COLLECTION_GROUP). All fields are scalar ordered fields; no ARRAY_CONTAINS modes. A=ASCENDING, D=DESCENDING. Firestore adds **name** using the final ordered field direction. Products explicitly query document ID descending, matching the implicit **name** D on these definitions.

Collection	Fields in index order	Query
products	status A, createdAt D, **name** D (implicit)	newest / legacy public list without category
products	status A, featured D, createdAt D, **name** D (implicit)	featured without category
products	status A, pricePaise A, createdAt D, **name** D (implicit)	price-asc without category
products	status A, pricePaise D, createdAt D, **name** D (implicit)	price-desc without category
products	status A, categoryId A, featured D, createdAt D, **name** D (implicit)	featured with category
products	status A, categoryId A, pricePaise A, createdAt D, **name** D (implicit)	price-asc with category
products	status A, categoryId A, pricePaise D, createdAt D, **name** D (implicit)	price-desc with category
products	status A, categoryId A, createdAt D, **name** D (implicit)	newest / legacy public list with category
categories	slug A, enabled A, **name** A (implicit)	Enabled category slug lookup before browse
orders	reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Preserved earlier aggregate definition (not currently queried) without date range
orders	createdAt A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Preserved earlier aggregate definition (not currently queried) with date range
orders	paymentStatus A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Preserved earlier aggregate definition (not currently queried) without date range
orders	paymentStatus A, createdAt A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Preserved earlier aggregate definition (not currently queried) with date range

Collection	Fields in index order	Query
orders	status A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Legacy wrong-order definition; superseded by REPORT_INDEX_AUDIT.md
orders	status A, createdAt A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Legacy wrong-order definition; superseded by REPORT_INDEX_AUDIT.md
orders	status A, paymentStatus A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Legacy wrong-order definition; superseded by REPORT_INDEX_AUDIT.md
orders	status A, paymentStatus A, createdAt A, reportingPaise A, collectedPaise A, refundPaise A, **name** A (implicit)	Legacy wrong-order definition; superseded by REPORT_INDEX_AUDIT.md
orders	status A, createdAt A, **name** A (implicit)	Independent per-status count with date bounds
orders	status A, paymentStatus A, **name** A (implicit)	Independent active-status/payment-due count without date bounds
orders	status A, paymentStatus A, createdAt A, **name** A (implicit)	Independent active-status/payment-due count with date bounds
inventoryAudit	productId A, createdAt D, **name** D (implicit)	Per-product stock audit history

Browse bounds: status == Published; optional categoryId equality after enabled slug resolution; featured D or pricePaise A/D, then createdAt D and document ID D. Newest omits the first sort field. Cursor is an existing document snapshot, not an offset. Server scans at most 200 index-ordered rows and returns at most 50 matches. Search, colors (including variant names), maximum price, in-stock and sale filters run on this bounded server page and do not add index fields. A page may contain no matches and still have a next cursor. Current category visibility is rechecked before presentation. No browser sorting or unbounded fallback was added.

reports:get runs seven status-filtered count/sum aggregates and an active-status IN plus paymentStatus=due aggregate, each paired with a separate count for completeness in one read-only transaction snapshot, so concurrent orders cannot cause a spurious mismatch. Optional from/to use inclusive createdAt timestamp bounds, independent of collection/refund timestamps. Aggregates include reportingPaise, collectedPaise and refundPaise in the same index. Invalid/reversed/out-of-Firestore-range dates now return invalid (400). Inventory page scans use createdAt D/document ID D with a 200-row bound; they are not global stock totals.

Original index-publication guidance - reporting portion superseded

For the current reporting failure, use only the four exact additional definitions/links in REPORT_INDEX_AUDIT.md. All seven owner-created orders indexes are READY; do not recreate them. The original broad publication guidance below is historical context, not the current remediation list.

1. Sign into Firebase Console, choose project **parinaya-silks**, Firestore Database **(default)**, Indexes, Composite, Add index. Use the collection, COLLECTION scope and field directions in the table. Prioritize featured without category, the four currently queried combined-report aggregate variants and the three independent count indexes. A count filtered only by status uses its automatic single-field index. Create the other browse/category variants for supported sorts/filters. Do not create the implicit **name** manually when the console supplies it.
2. Alternatively, from this checkout after comparing deployed indexes and retaining any remote-only definitions, run `npx.cmd firebase deploy --only firestore:indexes --project parinaya-silks`. firebase.json targets firestore.indexes.json; no named database override means (default). Review the CLI plan; do not accept deletion of remote-only indexes. This is a future owner action, not authorization to execute it in this task.
3. Wait for every required index to finish building and show **Enabled/Ready** before testing dependent queries. Vercel redeployment alone does not create Firestore indexes.
4. Future errors now retain a project-bound console create_composite URL in server diagnostics, when Firestore supplies one. Inspect that trusted link to distinguish the exact missing report variant. Public responses expose only safe status/code/request ID.

Legacy schema audit / migration plan (no live changes)

Existing new product writes include featured (boolean), pricePaise (integer) and createdAt (Firestore timestamp). Ordering excludes records missing a sorted field; a missing featured field can hide an old Published product specifically in featured browsing. A missing createdAt excludes it from all these browse sorts and inventory scans; missing pricePaise excludes price sorting. Missing/invalid category or schema still needs owner review; visibility must not be bypassed.

Before any migration, take an authorized backup/export. Audit an offline export or a separately authorized read-only, paged admin scan ordered by document ID (not createdAt) so missing timestamps are included. Check product status/category/featured/pricePaise/createdAt/schemaVersion and order status/paymentStatus/createdAt/reportingPaise/collectedPaise/refundPaise. Keep any exported customer/order data private and out of Git/logs. Produce only counts and a protected owner review list. No current production export or audit was performed, so whether a migration is needed remains unverified.

Do not invent creation dates, convert ambiguous price units, assume featured=false, mark COD collected, or guess refunds. Propose minimal missing-field patches only from authoritative existing records; explicitly review ambiguous cases. For orders, reportingPaise must agree with validated totalPaise, collectedPaise with the actual collection record and refundPaise with actual refund entries. Combined count/sum aggregates only include documents containing all aggregated fields and ignore nonnumeric sum values; legacy incomplete records can therefore understate metrics. Reports now compare each combined aggregate count with an independent count of the same filtered orders and fail safely if missing aggregate fields excluded orders. Nonnumeric-but-present legacy values still require the offline schema audit. Treat dashboard completeness as unverified until this audit passes. Test proposed patches on a restored local emulator copy first. Future apply must use reviewed IDs, expected revisions/update times and transactions, preserve unrelated fields, never replace/delete documents and be safe to rerun. The named owner product is excluded from any automatic patch plan.

Navigation and shared editors

Sidebar order	Route	Implementation
1. Dashboard	/admin/dashboard	COD reports + independent bounded inventory preview
2. Products	/admin/products	Existing ProductManager, including edit/publish/images/variants
3. Categories	/admin/categories	Existing category manager
4. Orders	/admin/orders	Existing COD order workspace
5. Customers	/admin/customers	Existing profile directory + order history
6. Inventory	/admin/inventory	Shared low-stock component; links to existing per-product reasoned adjustments/audit and all products
7. Marketing / Sale Campaigns	/admin/marketing	Existing coupons; /admin/coupons remains a working active-menu alias
8. Banners	/admin/banners	Shared AdminContent editor, banner fields only
9. Website Content	/admin/website-content	Same stored document/editor: featured product/category references, policies and return-request configuration
10. Contact Information	/admin/contact-information	Same stored document/editor: approved contact/social fields
11. About Us	/admin/about	Explicit unavailable: no brand-story editor/data model exists
12. Customer Enquiries	/admin/enquiries	Explicit unavailable: no genuine enquiry service exists
13. Settings	/admin/settings	Existing shipping + Login Page Appearance

View Public Store and Logout remain separate utilities; notification bell remains accessible. Existing /admin, /admin/login, /admin/coupons, product and order detail URLs remain supported. Both menus share the same order; active state includes aliases and detail routes. The sidebar scrolls independently at shorter heights.

The content pages load/save one revision-controlled siteSettings/storeContent document through the existing verified-media API. Section views do not create conflicting settings copies. Existing Save/Cancel and unsaved navigation checks remain. The previous combined editor has moved out of Settings; its underlying API/document and permissions are unchanged.

Local fixes and validation boundary

- Unknown catalog counts now say Loading sarees / Count unavailable. Zero appears only on a successful empty page. A + marks a loaded count with more results.
- Failed browse requests do not restart on focus/visibility/timers; manual retry preserves selections. Identical public catalog requests share one in-flight promise, with no response cache. The parent storefront no longer requests a redundant unfiltered catalog while Shop/Collections owns browse.
- Existing generation guards reject stale responses. Failed reports/inventory have section-specific messages and independent retry; no failed metric is converted to zero. Card hierarchy shows only server report amounts/statuses. Delivered order value stays distinct from recorded COD collections.
- Trusted index creation URLs are server-side only, restricted to the exact Firebase console host/project/path and one create_composite parameter. Other exception messages/credentials/headers/bodies are never serialized.
- Dual administrator authorization, customer isolation, COD-only orders, shipping calculation and current rules are unchanged. No Firestore rule publication is needed for these local changes.

Verification results are recorded at the end after running tests. Emulator success does not establish production index existence/readiness. Production remediation remains outstanding until the owner publishes indexes, waits for Ready, publishes the local code and retests actual browse/report actions with refreshed admin authorization.

Official references: https://firebase.google.com/docs/firestore/query-data/index-overview and https://firebase.google.com/docs/firestore/query-data/aggregation-queries.

Verification completed locally

- `npm.cmd run lint`: pass (TypeScript noEmit).
- `npm.cmd test`: 71/71 pass, including operation-stage HTTP browse/report missing-index responses, project-bound diagnostic link redaction, actual query/index contracts and safe reorganized destinations.
- `npm.cmd run build`: pass; existing large-chunk advisory remains.
- All 10 emulator suites passed across the full runner and focused reruns: administrator rules, category rules, catalog CRUD, appearance, customer profiles, shopping, delivery, shipping, COD orders and business APIs. Business was rerun on the final read-only report snapshot implementation, including missing aggregate field rejection, invalid date bounds, all sorts/category combinations and tied-value pagination without gaps/duplicates.
- All 11 browser suites passed across the same local reruns: customer authentication, appearance, administrator access, categories, products, shopping, delivery, shipping, content/coupons, new recovery/navigation and COD. Existing suites cover six widths (320/375/390/768/1024/1440); the recovery suite additionally checks 320/390/768/1440.
- Recovery browser checks: one initial browse request, no focus retry after failure, retry keeps selected sort, unknown versus successful-empty counts, late failed sort response cannot overwrite a newer successful result, independent reports/inventory failures, exact sidebar order, all 13 routes/direct refresh, coupons alias, active navigation, Back and mobile drawer closure/highlighting.
- Packaged-function check: pinned Vercel builder 24.0.3 produced a nodejs24.x artifact, invoked successfully by local Node 22.22.2 without a tsx loader. This does not verify hosted Node 24 or production Firestore indexes.
- Desktop and 320px mobile screenshots were visually inspected. No overlap/horizontal overflow was observed. Ignored artifacts: `.cache/catalog-recovery/`.
- `git diff --check`: pass. Existing index definitions and fieldOverrides preservation were checked against HEAD; exactly three definitions added.

During validation, a new test expected Categories instead of the existing Manage Categories heading, and an inventory assertion reused an intentionally incomplete fixture. Those test assertions were corrected and the affected suites rerun successfully. The full runner was stopped by those assertions; the successful final focused runner reused the same suite implementation and completed the remaining tests. No production test or deployment occurred.

Remaining owner production checklist

1. Compare remote indexes with the manifest, preserve remote-only definitions, publish the required indexes to parinaya-silks/(default), then wait for all dependent indexes to finish building.
2. Review the offline legacy-schema audit/migration plan before asserting reporting completeness. No migration is proven necessary from the supplied logs, and none was executed.
3. Commit/push/deploy the reviewed local changes through the owner's normal process. No additional environment variables, rules or serverless endpoints are introduced by this change.
4. Test the actual featured GET, all sorts/category/filter combinations and pagination. Test reports:get with/without dates, independent inventory, retry/error states, new navigation/direct URLs, existing COD and authentication flows. Confirm the named owner product is intact. Do not create or mutate live products merely to test listing.
5. Review request IDs/logs for any remaining failure. Both supplied failures are confirmed index errors; the exact missing reporting member and deployed index readiness remain unverified because no creation URL or remote index inventory was supplied. Emulator and packaged-function success are not proof that the hosted 503 is resolved.

Final follow-up: both COD orders and business API emulator suites passed again after the read-only transaction change. The COD regression verifies recorded collections of 109900 paise, manual refunds of 1000 paise and zero due in its synthetic fixture. No test listeners remain on the application ports.

Changed files

- Query/index and diagnostics: `firestore.indexes.json`, `server/business.ts`, `server/diagnostics.ts`.
- Storefront request/state handling: `src/App.tsx`, `src/products/client.ts`, `src/products/hooks.ts`, `src/views/ShopView.tsx`.
- Administrator navigation, shared sections and reporting: `src/admin/AdminLayout.tsx`, `src/admin/AdminPanel.tsx`, `src/admin/accessPolicy.ts`, `src/content/AdminContent.tsx`, `src/orders/AdminBusiness.tsx`, `src/orders/ui.tsx`.
- Validation: `package.json`, `scripts/admin-emulator-suite.ts`, `scripts/business-browser.test.py`, `scripts/business-emulator.test.ts`, `scripts/catalog-handler.test.ts`, `scripts/catalog-indexes.test.ts`, `scripts/catalog-recovery-browser.test.py`, `src/admin/accessPolicy.test.ts`.

- Documentation: this file and `docs/COMPLETION.md`.